

Lab09. Running contenerized application in ECS and ElasticBeanstalk

Cloud Programming (W04IST–SI4527G)

Wojciech Thomas

Spring 2026

1 Learning goals

After this assignment you should be able to:

1. Publish your contenerized application to AWS ECR container registry.
2. Deploy your application from ECR into ECS (using Fargate backend).

2 Exercises

2.1 Prerequisites

1. Download demo.zip folder from ePortal. Upload file to Cloud9 and unpack it. The folder contains sample Python web application.
2. Create a new `main.tf` file that creates new VPC for this lab.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.1"
    }
  }
  required_version = ">= 1.2.0"
}

provider "aws" {
  region = "us-east-1"
}

module "my_vpc" {
  source      = "terraform-aws-modules/vpc/aws"
  name        = "ecs-lab"
  cidr        = "10.0.0.0/16"
  azs         = ["us-east-1a"]
  public_subnets = ["10.0.101.0/24"]
  tags = {
```

```

    Terraform    = "true"
    Environment  = "dev"
  }
}

```

3. Create Security Group that allow incoming traffic on port 8080, and all outgoing traffic:

```

resource "aws_security_group" "allow_http" {
  name            = "allow_http"
  description     = "Allow HTTP inbound traffic and all outbound traffic"
  vpc_id         = module.my_vpc.vpc_id
  tags = {
    Name = "Allow HTTP"
  }
}

resource "aws_vpc_security_group_egress_rule" "allow_all_traffic_ipv4" {
  security_group_id = aws_security_group.allow_http.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "-1" # all ports
}

resource "aws_vpc_security_group_ingress_rule" "allow_http" {
  security_group_id = aws_security_group.allow_http.id
  cidr_ipv4         = "0.0.0.0/0"
  ip_protocol       = "tcp"
  from_port         = 8080
  to_port           = 8080
}

```

4. Save and apply Terraform configuration. Verify if both: VPC and Security Group were created.

2.2 Configure ECR

ECR is Amazon AWS managed service keeping the registry of container images. For each container image, you create a separate repository. Single repository can keep multiple versions of the particular container image.

1. Add to `main.tf` module to create ECR:

```

resource "aws_ecr_repository" "pythonflasksample" {
  name = "pythonflasksample"

  image_scanning_configuration {
    scan_on_push = true
  }

  tags = {
    Name = "PythonFlaskSample"
  }
}

```

2. Create ECR. Open AWS console and check, if the ECR registry is created.

2.3 Push container image to ECR

As the main goal of Terraform is building infrastructure, we will push container images manually to the registry.

1. In terminal, change directory to `python-flask-sample` folder.
2. Review the content of `Dockerfile` .
3. Build the container image (dot “.” is the part of the command!):

```
docker build -t pythonflasksample:v1 -t pythonflasksample:latest .
```

Pay attention to two tags: `latest` and version.

4. List built images: `docker images`

Find created images.

5. Log in Docker to ECR (Elastic Container Registry):

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin  
↪ <ecr-dns>
```

`<ecr-dns>` should look like: `901425240435.dkr.ecr.us-east-1.amazonaws.com`. You can find the number using command: `aws ecr describe-registry`. You can also click the blue button “View push commands” to find the command.

6. Add new tags (labels) to the container image:

```
docker tag pythonflasksample:latest <ecr-dns>/pythonflasksample:latest  
docker tag pythonflasksample:latest <ecr-dns>/pythonflasksample:v1
```

List container images and find the new tags.

We assign to the image two tags: `latest` and version. The newest version will always have two tags. If you do not provide version label, the `latest` will be assumed.

7. Push container image to ECR – compare the speed of both push operations:

```
docker push <ecr-dns>/pythonflasksample:latest  
docker push <ecr-dns>/pythonflasksample:v1
```

Open AWS console, switch to ECR service, and verify if the container image is available in the registry.

2.4 Configure ECS using Fargate cluster

ECS is the AWS service that provides UI and API to manage application running in containers. It can run containers in Fargate (managed Docker cluster) or on your EC2.

1. Add to the `main.tf` command to configure ECS cluster:

```
resource "aws_ecs_cluster" "ecs_pfs" {
  name = "ecs_pfs"
}
```

Apply configuration and verify if ECS cluster has been created. `pfs` is an abbreviation of Python Flask Sample.

2. Add the command to build ECS Service:

```
resource "aws_ecs_service" "ecs_svc_pfs" {
  name           = "ecs_svc_pfs"
  cluster        = aws_ecs_cluster.ecs_pfs.id
  task_definition = aws_ecs_task_definition.ecs_task_pfs.arn
  launch_type    = "FARGATE"
  network_configuration {
    subnets      = [module.my_vpc.public_subnets[0]]
    assign_public_ip = true
    security_groups = [aws_security_group.allow_http.id]
  }
  desired_count = 1
}
```

ECS Service will keep our container. Container will be started in the managed Docker cluster named *Fargate*.

Pay attention, that you have to configure Security Group for the *service*, and not for the container.

3. Create ECS task definition. Task definition shows to AWS how to run container inside ECS Service:

```
resource "aws_ecs_task_definition" "ecs_task_pfs" {
  family              = "ecs_task_pfs"
  network_mode        = "awsvpc"
  requires_compatibilities = ["FARGATE"]
  memory              = "1024"
  cpu                 = "512"
  task_role_arn       = "<labrole-arn>"
  execution_role_arn  = "<labrole-arn>"
  container_definitions = <<-EOF
  [
    {
      "name": "pfs-container",
      "image": "<ecs-dns>/pythonflasksample:latest",
      "memory": 1024,
      "cpu": 512,
      "essential": true,
      "portMappings": [
        {
          "containerPort": 8080,
          "hostPort": 8080
        }
      ],
      "environment": [
```

```

    {
      "name": "USER_NAME", "value": "Bob"
    }
  ]
}
EOF
}

```

Open IAM service, and in the Roles find ARN of `LabRole`. It should look like: `arn:aws:iam::90142524532:role/LabRole`.

Pay attention, how to use environment variables to configure application running in the container.

4. Save `main.tf` file, validate its content, reformat it (`terraform fmt`), plan and apply.
5. Open AWS console, switch to ECS. Open Service, then Task. Find public IP address. Open in the browser Python web application running in the container.

2.5 Configure load balancer

As containers are ephemeral, their public IP address can change. In order to allow one, constant IP address (endpoint) for your application, you can deploy a load balancer (or reverse proxy) at the front of your container.

1. At the first step you will create a load balancer:

```

resource "aws_lb" "app_lb_pfs" {
  name           = "app-lb-pfs"
  internal       = false
  load_balancer_type = "application"
  security_groups = [aws_security_group.allow_http.id]
  subnets       = module.my_vpc.public_subnets

  enable_deletion_protection = false
}

```

As load balancer requires at least two subnets in different availability zones to work, you have to update the definition of your VPC. Change both: availability zones and IP subnets to:

```

module "my_vpc" {
  ...
  azs           = ["us-east-1a", "us-east-1b"]
  public_subnets = ["10.0.101.0/24", "10.0.102.0/24"]
  ...
}

```

2. Now add a target group. This object checks the health of containers running in ECS and keeps a list of healthy containers:

```

resource "aws_lb_target_group" "app_tg_pfs" {

```

```

port      = 8080
protocol  = "HTTP"
vpc_id    = module.my_vpc.vpc_id
target_type = "ip"
health_check {
  path           = "/"
  protocol       = "HTTP"
  matcher        = "200"
  interval       = 30
  timeout        = 5
  healthy_threshold = 2
  unhealthy_threshold = 2
}
}

```

Pay attention, how the availability of the container (health) is verified.

3. Create the listener (public endpoint) for the load balancer:

```

resource "aws_lb_listener" "http_listener_pfs" {
  load_balancer_arn = aws_lb.app_lb_pfs.arn
  port              = 8080
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.app_tg_pfs.arn
  }
}

```

4. Finally, you have to connect your ECS service with target group. In the definition of the `ecs_svc_pfs` service, add the following code just before the closing bracket:

```

load_balancer {
  target_group_arn = aws_lb_target_group.app_tg_pfs.arn
  container_name   = "pfs-container"
  container_port   = 8080
}

depends_on = [aws_lb_listener.http_listener_pfs]

```

5. Validate, format and apply your Terraform configuration.
6. In AWS console open EC2 service, then scroll the left panel to “Load Balancers” section.
7. Find your load balancer and select it. At the bottom search “DNS name” that looks like: `app-lb-pfs-1520283452.us-east-1.elb.amazonaws.com`. In the browser, open address: `http://<lb-dns-name>:8080/`.

You should see your Python app working in the ECS.

8. In AWS console review the ECS cluster, task definition, running container.
9. Open the “Task definitions” page, and select `ecs_task_pfs`. Select “Create new revision” (blue

button at the top). Find “Environment variables” section. Change the value of the `USER_NAME` variable. Scroll down and click “Create”.

10. Click “Deploy” (blue button at the top) and select “Update service”. Scroll down and click “Update”.
11. Wait until “Tasks” are in “Running” status. Open the page with your app. Has the message changed? It should!